



合肥师范学院
HEFEI NORMAL UNIVERSITY

实验报告

课程名称: web 程序设计

学院: 计算机与人工智能学院

专业: 计算机科学与技术

学号: 2410211124

姓名: 朱品赞

指导教师: 张由明

2025 年 12 月 27 日

实验四 会话技术、过滤器技术实验

一、实验目的

- 1. 掌握 HttpSession 会话技术，理解会话的生命周期和作用域，学会使用 HttpSession 进行用户状态管理
- 2. 掌握 Servlet 过滤器技术，理解过滤器的工作原理和应用场景，学会实现 Filter 接口编写自定义过滤器
- 3. 实现基于会话的身份认证和权限控制，理解过滤器和会话的配合应用

二、实验内容及步骤

2.1 HttpSession 会话技术

HttpSession 工作原理

会话的完整生命周期：



HttpSession 核心方法

方法	说明	用途
setAttribute(String name, Object value)	存储属性	在会话中存储数据
getAttribute(String name)	获取属性	从会话中读取数据
removeAttribute(String name)	删除属性	移除会话中的某个属性
invalidate()	销毁会话	清除会话及所有属性
getCreationTime()	获取创建时间	获取会话创建的时间戳

方法	说明	用途
<code>getLastAccessedTime()</code>	获取最后访问时间	获取会话最后被访问的时间
<code>getMaxInactiveInterval()</code>	获取最大空闲时间	获取会话超时时间（默认30分钟）

项目中的会话应用

场景 1：用户登录时创建会话

```
public void login(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    String username = req.getParameter("username");
    String password = req.getParameter("password");

    SysUser sysUser = sysUserService.login(username, password);

    if (sysUser != null) {
        // 获取会话对象
        HttpSession session = req.getSession();

        // 在会话中存储已登录用户信息
        session.setAttribute("loginUser", sysUser);

        // 重定向到任务管理页面
        resp.sendRedirect(req.getContextPath() + "/showSchedule.html");
    } else {
        resp.sendRedirect(req.getContextPath() + "/loginError.html");
    }
}
```

关键点：

- `req.getSession()` - 获取当前会话，如果不存在则创建新会话
- `session.setAttribute("loginUser", sysUser)` - 将用户对象存储在会话中
- 存储的数据在整个会话生命周期内有效

场景 2：访问受保护资源时检查会话

```
public void add(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    HttpSession session = req.getSession();
```

```

// 从会话中获取登录用户
SysUser loginUser = (SysUser) session.getAttribute("loginUser");

// 如果用户未登录，重定向到登录页面
if (loginUser == null) {
    resp.sendRedirect(req.getContextPath() + "/login.html");
    return;
}

String title = req.getParameter("title");
SysSchedule schedule = new SysSchedule();
schedule.setUid(loginUser.getUid()); // 使用会话中的用户 ID
schedule.setTitle(title);
schedule.setCompleted(0);

boolean result = sysScheduleService.addSchedule(schedule);

if (result) {
    resp.sendRedirect(req.getContextPath() + "/showSchedule.html");
} else {
    resp.getWriter().write("新增日程失败");
}
}

```

关键点：

- `session.getAttribute("loginUser")` - 获取会话中存储的用户对象
- 检查用户是否为 null 来判断是否已登录
- 使用会话中的用户 ID 关联业务数据

场景 3：用户登出时销毁会话

```

public void logout(HttpServletRequest req, HttpServletResponse resp) throws
IOException {
    HttpSession session = req.getSession();

    // 销毁会话，清除所有会话属性
    session.invalidate();

    // 重定向到登录页面
    resp.sendRedirect(req.getContextPath() + "/login.html");
}

```

关键点：

- `session.invalidate()` - 完全销毁会话，清除所有数据和 JSESSIONID
- 调用后该会话的所有属性都无法访问

2.2 Servlet 过滤器技术

Filter 接口说明

Filter 接口的三个核心方法：

```
public interface Filter {  
    // 初始化过滤器，应用启动时调用一次  
    void init(FilterConfig filterConfig) throws ServletException;  
  
    // 过滤逻辑处理，每次请求都会调用  
    void doFilter(ServletRequest request, ServletResponse response,  
                  FilterChain chain) throws IOException, ServletException;  
  
    // 销毁过滤器，应用关闭时调用一次  
    void destroy();  
}
```

FilterChain 的作用：

- 表示过滤器链
- 调用 `chain.doFilter(request, response)` 将请求传递给下一个过滤器或目标资源
- 如果不调用 `chain.doFilter`，则请求链中断

项目中的登录过滤器实现

```
package com.zpz.schedule.filter;  
  
import com.zpz.schedule.pojo.SysUser;  
  
import javax.servlet.*;  
import javax.servlet.annotation.WebFilter;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
import javax.servlet.http.HttpSession;  
import java.io.IOException;  
  
@WebFilter({" /showSchedule.html", " /schedule/*"})  
public class LoginFilter implements Filter {  
  
    @Override
```

```

    public void init(FilterConfig filterConfig) throws ServletException {
        // 过滤器初始化，通常用于加载资源
    }

    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse
servletResponse,
                        FilterChain filterChain) throws IOException,
ServletException {
        // 类型转换为 HTTP 对象
        HttpServletRequest req = (HttpServletRequest) servletRequest;
        HttpServletResponse resp = (HttpServletResponse) servletResponse;

        // 获取当前会话
        HttpSession session = req.getSession();

        // 从会话中获取已登录用户
        SysUser loginUser = (SysUser) session.getAttribute("loginUser");

        // 如果用户未登录
        if (loginUser == null) {
            // 重定向到登录页面，中止请求
            resp.sendRedirect(req.getContextPath() + "/login.html");
            return; // 不继续执行过滤器链
        }

        // 用户已登录，继续执行过滤器链
        filterChain.doFilter(req, resp);
    }

    @Override
    public void destroy() {
        // 过滤器销毁时的清理工作
    }
}

```

过滤器的配置和执行流程

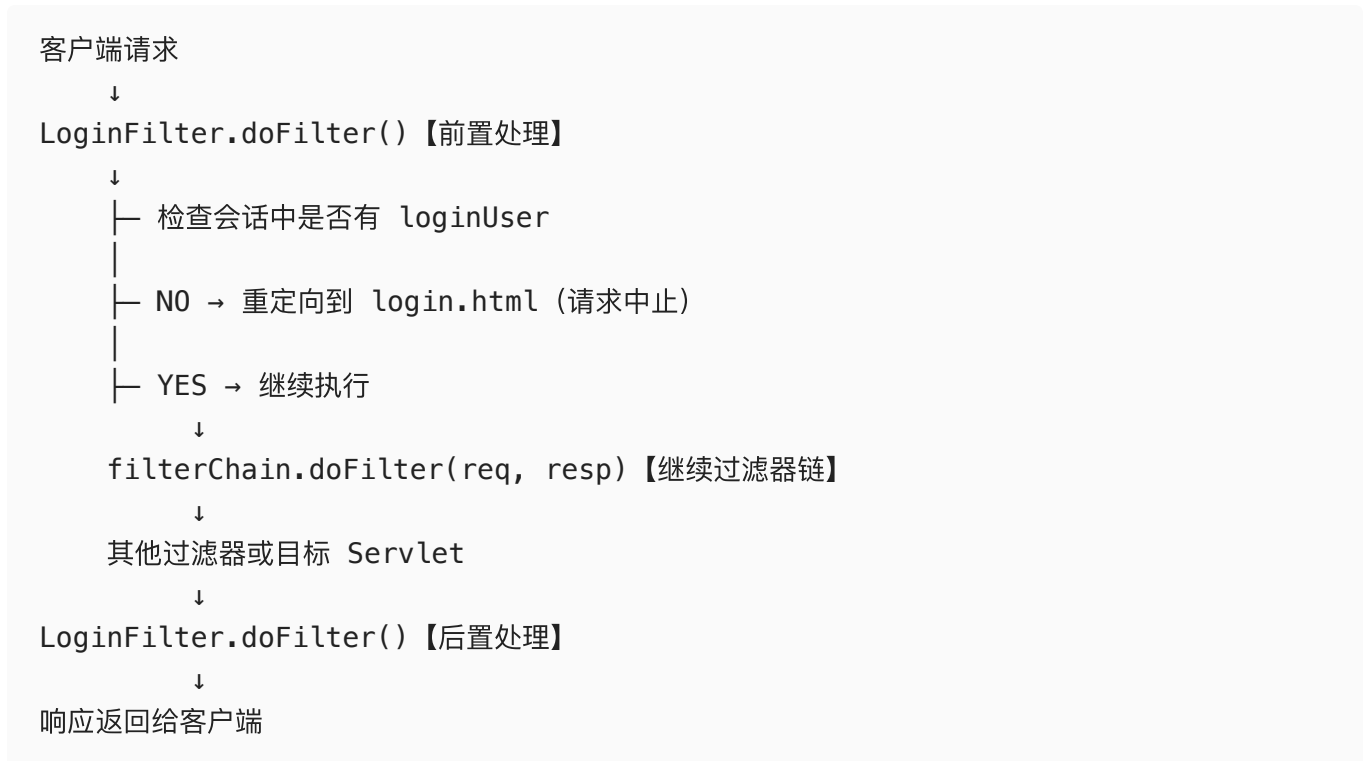
@WebFilter 注解的使用：

```
@WebFilter({" /showSchedule.html", " /schedule/*"})
```

URL 模式	说明	示例
/showSchedule.html	精确匹配	仅拦截 showSchedule.html

URL 模式	说明	示例
/schedule/*	路径匹配	拦截所有 /schedule/ 开头的请求
/*	通配符	拦截所有请求
*.jsp	扩展名匹配	拦截所有 JSP 页面

过滤器执行流程：



过滤器链执行顺序：

当配置多个过滤器时，执行顺序为：



2.3 常见过滤器应用场景

身份认证过滤器

```

@WebFilter({" /showSchedule.html", "/schedule/*" })
public class LoginFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                        FilterChain chain) throws IOException,
                        ServletException {
  
```

```

        HttpSession session = ((HttpServletRequest) request).getSession();
        if (session.getAttribute("loginUser") == null) {
            ((HttpServletResponse) response).sendRedirect("/login.html");
            return;
        }
        chain.doFilter(request, response);
    }
}

```

字符编码过滤器

```

@WebFilter("/*")
public class CharacterEncodingFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException,
        ServletException {
        request.setCharacterEncoding("UTF-8");
        response.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");
        chain.doFilter(request, response);
    }
}

```

请求日志过滤器

```

@WebFilter("/*")
public class LogFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain) throws IOException,
        ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        long startTime = System.currentTimeMillis();

        System.out.println("请求 URL: " + req.getRequestURI());
        System.out.println("请求方法: " + req.getMethod());

        chain.doFilter(request, response);

        long duration = System.currentTimeMillis() - startTime;
        System.out.println("处理耗时: " + duration + "ms");
    }
}

```


三、实验结果

3.1 身份认证完整流程

1. 用户在登录页面输入用户名和密码
↓
2. 提交到 POST /user/login
↓
3. SysUserController.login() 验证用户身份
 - |
 - |─ 调用 SysUserService.login(username, password)
 - |─ 查询数据库: SELECT uid, username, user_pwd FROM sys_user WHERE username = ?
 - |─ MD5 密码对比: MD5(输入密码) = 数据库密码
 - |
 - |─ 验证成功
↓
4. 创建会话, 存储用户信息
 - |─ HttpSession session = req.getSession()
 - |─ session.setAttribute("loginUser", sysUser)
 - |─ 将 JSESSIONID 写入 Cookie, 发送给客户端
↓
5. 重定向到 /showSchedule.html
 - |─ HTTP 302 重定向
 - |─ 客户端再次请求 /showSchedule.html
 - |─ 请求中自动携带 Cookie (包含 JSESSIONID)
↓
6. LoginFilter 拦截请求, 检查会话中的 loginUser
 - |─ 从请求的 JSESSIONID 找到对应 HttpSession
 - |─ 调用 session.getAttribute("loginUser")
 - |─ loginUser != null → 继续执行
↓
7. 过滤器链继续, 目标 Servlet 返回任务管理页面
 - |─ 查询该用户的日程列表
 - |─ SELECT sid, uid, title, completed FROM sys_schedule WHERE uid = ?
 - |─ 返回 HTML 页面, 展示日程列表
↓
8. 浏览器显示任务管理页面

3.2 保护受限资源

被 @WebFilter 保护的资源:

资源	说明	需要登录
/showSchedule.html	任务管理页面	✓ 需要
/schedule/add	添加日程接口	✓ 需要
/schedule/delete	删除日程接口	✓ 需要
/schedule/update	修改日程接口	✓ 需要

不被保护的资源：

资源	说明	需要登录
/login.html	登录页面	✗ 不需要
/register.html	注册页面	✗ 不需要
/user/register	用户注册接口	✗ 不需要
/user/login	用户登录接口	✗ 不需要

3.3 多层防护设计

第一层防护：过滤器

- └ @WebFilter({"showSchedule.html", "/schedule/*"})
- └ 检查会话中的 loginUser
- └ 未登录则重定向到登录页面

↓

第二层防护：控制器方法

- └ public void add(HttpServletRequest req, HttpServletResponse resp)
- └ 再次检查会话中的 loginUser
- └ 确保双重验证

↓

第三层防护：业务逻辑

- └ SysScheduleService 中的业务操作
- └ 验证用户权限和数据合法性

防护效果：

- 1. 层级防护 - 多个验证点，避免单点风险
- 2. 数据隔离 - 每个用户只能访问自己的日程

- 3. 会话安全 - 通过 JSESSIONID 保证会话唯一性
- 4. 权限控制 - 过滤器 + 业务逻辑双重保障

3.4 会话和过滤器配合示例

用户退出流程：

1. 用户点击退出按钮
↓
2. 浏览器请求 /user/logout
↓
3. LoginFilter 检查会话
 - ├ session.getAttribute("loginUser") != null
 - └ 继续执行↓
4. SysUserController.logout() 处理退出
 - ├ HttpSession session = req.getSession()
 - ├ session.invalidate() // 销毁会话
 - └ 清除所有会话属性和 JSESSIONID↓
5. 重定向到登录页面
↓
6. 用户尝试再次访问 /showSchedule.html
 - ├ 浏览器发送的 JSESSIONID 对应会话已被销毁
 - ├ 服务器创建新的会话，但其中没有 loginUser↓
7. LoginFilter 检查会话
 - ├ session.getAttribute("loginUser") == null
 - └ 重定向到 /login.html↓
8. 成功退出并返回登录页面

四、常见问题

无